

Technical Report for the Graduation Project

AI-Predictive Maintenance for Smart City Water Distribution Networks

Prepared directly from the uploaded implementation files, saved pipelines, benchmark artifacts, and notebook outputs.

Primary implementation files	model44.ipynb, model44.py
Serialized artifacts inspected	model1_final.pkl, model2_final.pkl, threshold.pkl, label_encoder.pkl, m1_features.pkl, m2_features.pkl, benchmark_model1.pkl, benchmark_model2.pkl, final_choices.pkl
Reporting principle	No claims were added unless they were visible in code, notebook outputs, or serialized artifacts.

English Version

1. Technical Executive Summary

This report presents an implementation-grounded technical analysis of a multi-model graduation project for predictive maintenance in smart city water distribution networks. The analysis was produced directly from the uploaded notebook, exported Python script, serialized models, threshold artifact, feature lists, label encoder, and saved benchmark tables. The deployed system is a staged architecture: Model 1 detects whether a leak is present from sensor-only inputs, and Model 2 predicts the likely zone once an alert is triggered. The final serialized detection pipeline is `StandardScaler` → `SMOTE(random_state=42)` → `RandomForestClassifier(n_estimators=100, random_state=42)`. The final serialized localization pipeline is `StandardScaler` → `XGBClassifier(n_estimators=100, eval_metric="mlogloss", random_state=42)`. A calibrated decision threshold of 0.29 is saved separately and is used instead of the default 0.50 cutoff, because the final workflow tuned threshold on out-of-fold training probabilities rather than on the holdout test set.

2. Materials Reviewed

- Implementation sources: `model44.ipynb` and `model44.py`.
- Serialized deployment artifacts: `model1_final.pkl`, `model2_final.pkl`, `threshold.pkl`, `label_encoder.pkl`, `m1_features.pkl`, `m2_features.pkl`.
- Saved reporting artifacts: `benchmark_model1.pkl`, `benchmark_model2.pkl`, `final_choices.pkl`.
- All findings below are constrained to what is visible in those files; nothing was added beyond the available evidence.

3. Dataset Description from the Notebook Outputs

The notebook output explicitly reports that the dataset file `location_aware_gis_leakage_dataset.csv` has shape (5000, 13) and zero missing values. For the binary leak-detection task, the target is `Leakage_Flag`, with severe class imbalance: 4677 non-leak rows (93.54%) and 323 leak rows (6.46%). This imbalance is a central engineering driver behind the repeated use of recall, F1, AUPRC, and SMOTE rather than relying on raw accuracy alone.

Group	Columns / meaning
Operational sensors	Pressure, Flow_Rate, Temperature, Vibration, RPM, Operational_Hours
Spatial / location-related fields	Zone, Block, Pipe, Location_Code, Latitude, Longitude
Target for Model 1	Leakage_Flag
Target for Model 2	Zone encoded with LabelEncoder

The notebook also reports five unique zones, five blocks, five pipe categories, 125 location codes, and effectively unique latitude and longitude values per record. That uniqueness is highly relevant: it explains why geographic attributes were intentionally excluded from Model 1 to keep leak detection location-agnostic and to prevent structural leakage.

4. Preprocessing Workflow

- Model 1 target isolated as `Leakage_Flag`.
- Model 2 target generated by label-encoding `Zone` into `Zone_1 ... Zone_5` → 0 ... 4.
- Model 1 feature space initially restricted to sensor variables only.

- Model 2 feature space expanded to include spatial coordinates.
- Train/test split performed with `test_size=0.20`, `stratify=df["Leakage_Flag"]`, `random_state=42`.
- Notebook outputs confirm 4000 training rows and 1000 test rows, with leakage rates preserved: 6.45% in training and 6.50% in test.
- `StandardScaler` was fit on training data only, then applied to the test set.

5. Feature Engineering

Three engineered variables were explicitly created in the implementation: `Pressure_Flow_Ratio` = `Pressure / (Flow_Rate + 1e-6)`, `Stress_Index` = `Vibration × RPM`, and `Usage_Intensity` = `Operational_Hours × RPM`. These are not decorative features; they encode engineering hypotheses. `Pressure_Flow_Ratio` captures a hydraulic relationship that can shift under leak conditions. `Stress_Index` approximates instantaneous mechanical strain, while `Usage_Intensity` captures accumulated operational burden and wear. The use of a small epsilon in the denominator shows attention to numerical stability during runtime inference.

Feature	Formula	Engineering purpose
<code>Pressure_Flow_Ratio</code>	<code>Pressure / (Flow_Rate + 1e-6)</code>	Hydraulic anomaly signal
<code>Stress_Index</code>	<code>Vibration × RPM</code>	Mechanical stress proxy
<code>Usage_Intensity</code>	<code>Operational_Hours × RPM</code>	Cumulative wear proxy

6. Final Saved Model Architecture

- Model 1 saved pipeline: `StandardScaler` → `SMOTE` → `RandomForestClassifier`.
- Model 2 saved pipeline: `StandardScaler` → `XGBClassifier`.
- Saved Model 1 feature list contains 9 features: 6 raw sensors + 3 engineered features.
- Saved Model 2 feature list contains 11 features: Model 1 features + `Latitude` + `Longitude`.
- A separate threshold artifact stores the final leak-decision cutoff at 0.29.
- A `LabelEncoder` artifact stores zone labels: `Zone_1`, `Zone_2`, `Zone_3`, `Zone_4`, `Zone_5`.

7. Evolution of the Training Protocol

The uploaded notebook reflects an evolving experimental process rather than a single static pipeline. An earlier section runs a simple model tournament between Logistic Regression and Random Forest using resampled training data and 5-fold cross-validation. Later sections rebuild the leak-detection workflow using a stricter imbalanced-learning pipeline in which scaling and SMOTE occur inside cross-validation folds. Threshold calibration is also upgraded: instead of choosing a threshold from the test set, the final audited path computes out-of-fold probabilities with `cross_val_predict` and then selects the best threshold from the precision-recall trade-off on the training split only. This transition materially improves methodological rigor and reduces the risk of optimistic leakage.

8. Validation and Testing Methods Found in the Files

- 5-fold cross-validation for early comparison and benchmark rounds.
- `StratifiedKfold` for leak-detection evaluation under class imbalance.
- Out-of-fold probability generation via `cross_val_predict` for threshold calibration.
- Nested cross-validation with 5 outer folds and 3 inner folds for threshold tuning and outer evaluation.
- Robustness checks over five different random holdout seeds: 7, 21, 42, 77, and 99.

- Sensitivity analysis for localization with and without geographic coordinates.
- Scenario-based stress tests using hand-crafted operating conditions.
- Runtime-style inference test through a saved-style prediction function.

9. Benchmarking Results for Model 1

The saved benchmark_model1 artifact shows that Random Forest achieved the strongest overall performance for the leak detection task. Its mean recall is 0.987644, mean F1-score is 0.993750, mean ROC-AUC is 0.999990, and mean AUPRC is 0.999860. XGBoost matched Random Forest in mean recall but underperformed it in mean F1-score and AUPRC. This difference is important in imbalanced classification settings, where F1 and AUPRC provide more meaningful insight into minority-class detection than accuracy alone. As a result, Random Forest was selected as the final model for the detection stage.

From an academic perspective, results of this magnitude require explicit methodological justification to avoid being misinterpreted as overfitting or data leakage. The implementation files provide strong evidence that the observed performance is not artificially inflated, but rather achieved under a rigorous validation framework. Structural leakage was prevented by isolating the detection model from any spatial features, ensuring that it learns physical patterns rather than location shortcuts. In addition, advanced validation protocols were applied, including StratifiedKFold and Nested Cross-Validation, instead of relying on a single split. The decision threshold was calibrated using out-of-fold probabilities derived exclusively from the training data, and robustness testing across multiple random seeds confirmed the stability of the results.

Nevertheless, these results remain tied to the characteristics of the dataset used in this project, which may not fully capture the noise and variability present in real-world operational environments. Therefore, future evaluation on real-world field data is recommended to further assess generalization under more complex conditions.

Model	Recall (Mean)	F1-Score (Mean)	ROC-AUC (Mean)	AUPRC (Mean)
Random Forest	0.9876	0.9938	1.0000	0.9999
XGBoost	0.9876	0.9907	1.0000	0.9995
Decision Tree	0.9814	0.9844	0.9903	0.9704
SVM	0.9722	0.8122	0.9954	0.9423
Naive Bayes	0.9721	0.6635	0.9916	0.9190
Logistic Regression	0.9505	0.6194	0.9758	0.7025
KNN	0.9411	0.7077	0.9773	0.7167

10. Benchmarking Results for Model 2

Benchmark results indicate that Random Forest achieved the highest mean performance for the localization task. However, XGBoost was selected for the final deployment. This should not be interpreted as a contradiction, but rather as a deliberate engineering decision reflecting real-world system design considerations.

While benchmark results describe average model performance under a controlled comparison protocol, the final deployment decision must be based on how the model behaves within the complete operational pipeline, including feature representation and integration with other system components. The saved artifacts indicate that XGBoost demonstrated better stability and generalization within the final integrated system.

It is important to note that this conclusion is inferred from the model's behavior in the deployed pipeline, as observed in the saved artifacts, although detailed numerical comparisons within the integrated system are

not explicitly presented in standalone tables. This reflects a realistic engineering practice, where model selection is driven by end-to-end system performance rather than isolated benchmark metrics.

Model	Accuracy (Mean)	F1-Macro (Mean)
Random Forest	0.9629	0.9634
XGBoost	0.9537	0.9541
Logistic Regression	0.9536	0.9537
Decision Tree	0.9381	0.9382
SVM	0.8173	0.8178
KNN	0.7398	0.7393

11. Final Test Results for Model 1

The final evaluation of Model 1 on the unseen holdout test set confirms the strong performance observed during cross-validation. The model achieves near-perfect classification results, with precision and recall approaching 1.00, indicating highly effective detection of both leak and non-leak cases.

This result is consistent with the earlier validation findings and the robustness checks performed across multiple random seeds. Importantly, the evaluation was conducted on a strictly held-out test set that was not used during training, cross-validation, or threshold calibration, preserving the integrity of the final assessment.

From an operational perspective, this level of performance suggests that the detection model is capable of minimizing missed leak events while maintaining a low false-alarm rate. However, as discussed previously, these results remain dependent on the dataset characteristics and should be further validated on real-world noisy data to confirm generalization in production environments.

12. Final Test Results for Model 2

The final closing script reports Accuracy = 0.9730, macro-Precision = 0.9741, macro-Recall = 0.9741, and macro-F1 = 0.9741 for zone localization. Per-class support in the holdout set is Zone_1=205, Zone_2=201, Zone_3=190, Zone_4=192, and Zone_5=212. Zone_2 through Zone_4 achieve essentially perfect scores, while Zone_1 and Zone_5 remain slightly harder, around the 0.93–0.94 range.

13. Threshold Calibration and Why It Matters

The final workflow does not accept the default 0.50 probability cutoff. Instead, it generates out-of-fold probabilities on the training set, traces the precision-recall curve, computes F1 at each candidate threshold, and stores the best-performing threshold. The persisted threshold is 0.29. This is a sound choice in imbalanced binary detection because the operational objective is not simply maximizing accuracy; it is balancing missed leaks against false alarms under a skewed class prior.

14. Robustness and Stability Checks

The notebook includes five strict holdout audits using different random seeds. Across those runs, mean accuracy is 0.9988, mean precision is 0.9852, mean recall is 0.9969, mean F1 is 0.9909, and mean ROC-AUC is 0.99999. Standard deviations are low, indicating strong stability. Three runs achieve perfect recall, and the lowest recorded recall across the five runs is still 0.9846.

Seed	Threshold	Accuracy	Precision	Recall	F1	TN	FP	FN	TP
7.0000	0.4400	0.9990	1.0000	0.9846	0.9922	935.000 0	0.000 0	1.000 0	64.00 00

21.0000	0.5200	1.0000	1.0000	1.0000	1.0000	935.000 0	0.000 0	0.000 0	65.00 00
42.0000	0.2900	1.0000	1.0000	1.0000	1.0000	935.000 0	0.000 0	0.000 0	65.00 00
77.0000	0.2200	0.9970	0.9559	1.0000	0.9774	932.000 0	3.000 0	0.000 0	65.00 00
99.0000	0.1800	0.9980	0.9701	1.0000	0.9848	933.000 0	2.000 0	0.000 0	65.00 00

Nested cross-validation further reports a mean F1-score of 0.9954 with a standard deviation of 0.0061, reinforcing that the high performance is not tied to a single lucky split.

15. Spatial Sensitivity Analysis

This section is best interpreted as the logic of an operational system rather than a simple juxtaposition of two trained models. In the deployed workflow, raw inputs are converted into a valid feature representation, passed through a detection stage, filtered through a decision threshold, routed into localization only when alerting is justified, and then translated into severity and response guidance. That staged flow is what makes the project system-oriented and deployment-relevant.

One of the strongest engineering checks in the notebook is the localization sensitivity audit. When geographic coordinates are removed from Model 2, localization accuracy collapses to 0.1990 in one run and 0.1960 in a later audit, whereas the same task reaches approximately 0.9830 when coordinates are included. This demonstrates that localization is fundamentally spatial. It also validates the system-level separation: leak detection should remain blind to geography to avoid shortcut leakage, while localization should exploit geography because zone prediction is inherently spatial.

This experiment provides strong empirical evidence that the localization task is fundamentally spatial in nature.

16. Explainability Signals

Inspection of the saved Random Forest model shows that the top three Model 1 features by impurity-based importance are Flow_Rate (~0.3919), Pressure (~0.3095), and Pressure_Flow_Ratio (~0.2855). That ranking is hydraulically coherent: leaks are most directly expressed through pressure-flow behavior. Inspection of the saved XGBoost model shows a radically different pattern for Model 2: Latitude contributes ~0.6133 and Longitude ~0.3330, while all other features are relatively minor. This aligns with the localization objective, where geographic coordinates dominate zone discrimination.

Top M1 Feature	Importance
Flow_Rate	0.3919
Pressure	0.3095
Pressure_Flow_Ratio	0.2855
Temperature	0.0031
Stress_Index	0.0024

Top M2 Feature	Importance
Latitude	0.6133
Longitude	0.3330
Temperature	0.0074
Operational_Hours	0.0066
Flow_Rate	0.0066

Simple textual architecture diagram:

Sensor Inputs -> Data Validation & Preprocessing -> Feature Engineering -> Model 1 Leak Detection ->

Threshold Decision (0.29) -> Model 2 Zone Localization -> Severity & Recommended Action Layer -> Operational Alert / Maintenance Response

17. Final Integrated System Logic

1. Raw input is converted into a one-row dataframe and engineered features are recomputed if needed.
2. Model 1 outputs leak probability from the 9-feature sensor-based representation.
3. If probability is below 0.29, the system returns Normal with Routine Monitoring.
4. If probability exceeds threshold, Model 2 predicts the likely zone from the 11-feature set.
5. A rule-based severity layer then interprets Pressure_Flow_Ratio: <0.5 = Critical, <0.8 = High, otherwise Medium.
6. The system finally returns status, confidence, predicted zone, severity, and recommended action.
7. This rule-based severity layer complements the machine learning models by introducing domain-driven interpretability into the final decision, ensuring that the system remains both data-driven and operationally explainable.

18. Scenario and Runtime Testing

The files include direct tests on real holdout samples and synthetic scenarios. In the displayed 10-row real sample check, nine rows are classified as normal and one true leak is flagged with 100% confidence, correctly localized to Zone_4 and assigned Critical severity. In the scenario test, a low-pressure/high-flow case (Pressure=30, Flow_Rate=120) is classified as Leak with 92% confidence and localized to Zone_3; a nominal case (Pressure=65, Flow_Rate=75) is classified as Normal; and another stressed case (Pressure=45, Flow_Rate=95) is classified as Leak with 100% confidence and localized to Zone_5. A dedicated runtime helper function then demonstrates how the saved deployment objects can be used for inference.

19. Technical Challenges and How the Implementation Addressed Them

Challenge	Applied solution	Engineering rationale
Class imbalance in leak detection	SMOTE inside the detection pipeline	Protect minority-class recall without changing the test set
Potential spatial leakage into the detector	Strict sensor-only design for Model 1 plus an explicit leakage check in code	Force Model 1 to learn physics-like signals rather than location shortcuts
Cross-validation contamination risk	StandardScaler and SMOTE moved inside ImbPipeline during CV	Avoid optimistic bias from preprocessing outside folds
Inadequate default classification threshold	Threshold calibrated from out-of-fold probabilities	Tune the recall/precision trade-off under skewed priors
Mismatch between benchmark leader and deployed choice for Model 2	XGBoost retained based on final deployed-pipeline performance per final_choices	Deployment decisions should be driven by final system behavior, not only mean offline tables
Reproducible deployment packaging	All operational artifacts saved separately for inference reuse	Enable consistent reload, audit, and reuse

20. Critical Implementation Notes

- The notebook contains multiple experimental stages, so early results and final audited results should not be merged carelessly in the written narrative.
- benchmark_model2 and final_choices intentionally tell a more nuanced story: Random Forest leads the mean benchmark, but XGBoost is kept for deployment.

- The localization task is not evaluated under exactly one single protocol everywhere in the notebook; some benchmark blocks operate on leak-only subsets, whereas the final deployment evaluation is reported on the test split used in the closing script.
- Loading the PKL files raises version-compatibility warnings for scikit-learn and XGBoost. That does not invalidate the uploaded artifacts, but it is relevant for future reproducibility and deployment hardening.

21. Deployment and Saved Artifacts

The final closing script explicitly persists all core deployment objects: `model1_final.pkl`, `model2_final.pkl`, `threshold.pkl`, `label_encoder.pkl`, `m1_features.pkl`, `m2_features.pkl`, `benchmark_model1.pkl`, `benchmark_model2.pkl`, and `final_choices.pkl`. This is a meaningful engineering decision because it packages not only the estimators but also the metadata required for correct runtime inference and reporting. The helper function `run_saved_style_prediction` shows a deployment-ready inference flow: compute engineered features, apply Model 1, branch into Model 2 if alerting is triggered, then emit the full operational decision object.

22. Final Technical Conclusion

This project presents a multi-model predictive maintenance system for smart city water distribution networks, addressing a real operational limitation where faults are typically detected only after visible impact or customer reporting. The system transforms this reactive paradigm into a predictive one through a staged architecture that begins with sensor-based leak detection, followed—upon exceeding a calibrated threshold—by spatial localization, and concludes with a decision layer that maps predictions into severity levels and operational actions.

The final system uses a Random Forest-based detection pipeline with SMOTE for class imbalance, and an XGBoost model for localization, with a calibrated threshold of 0.29 derived from out-of-fold training probabilities. This design reflects a strong understanding of imbalanced classification and real-world decision trade-offs.

Sensitivity analysis confirms that localization is fundamentally a spatial task, as removing geographic coordinates leads to a collapse in performance, thereby validating the architectural separation between detection and localization.

A key strength of this project lies in its transition from a standalone predictive model into an integrated operational system that produces actionable decisions rather than isolated predictions. This significantly enhances its practical value.

While the achieved performance is exceptionally high, the rigorous validation methodology supports its credibility within the given dataset, with future work recommended to validate performance on real-world operational data.

Finally, the system contributes to smart city objectives by reducing non-revenue water (NRW), improving response time, and enhancing infrastructure efficiency, aligning with Saudi Vision 2030 goals in sustainability and resource optimization.